

11 Clustering

Markéta Tranberg, mlap@viauc.dk

April 2026

Table of Contents

- 1 Introduction to clustering
- 2 k -means
- 3 Agglomerative clustering
- 4 DBSCAN
- 5 Clustering algorithms overview
- 6 Code example

Clustering

- Unsupervised learning: unlabelled data
- Grouping similar pictures together
- Measure of similarity

Applications

- Reverse image search
- Customer segmentation and marketing (business groups)
- Species classification based on genetic traits
- Anomaly detection

Algorithms

- k -means
- DBSCAN
- Hierarchical clustering
- and many more...

Table of Contents

- 1 Introduction to clustering
- 2 k -means**
- 3 Agglomerative clustering
- 4 DBSCAN
- 5 Clustering algorithms overview
- 6 Code example

k -means

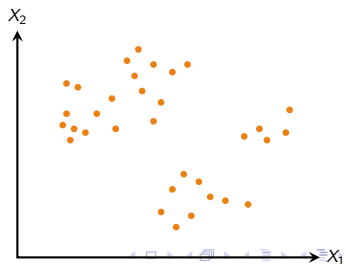
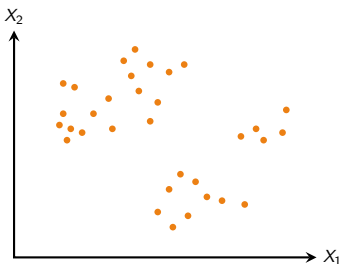
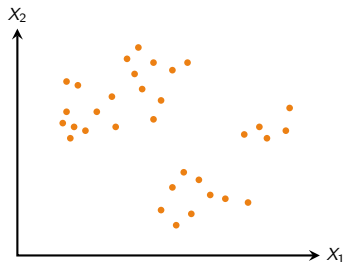
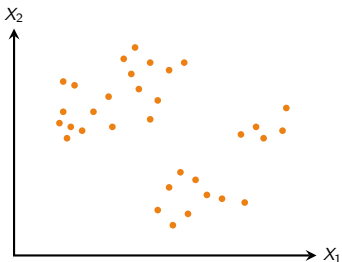
Do not confuse with k NN!

k -means: clustering

k NN: classification

k -means algorithm

- 1 Assign k random points as **centroids**
- 2 Group the data by their distance to centroids into clusters
- 3 Re-calculate the centroid to be in the centre of each cluster
- 4 Re-group the data
- 5 Repeat until nothing changes

k -means

Choice of initial centroids

Different initial centroids might result in different groups.

See code

Options for initial centroids

- Random
- Forgy
- k -means++
- Manual

Random partition

Randomly assign every datapoint to one of the k clusters, each initial centroid is in mean of those points.

- Pros: easy, cheap
- Cons: centroids are often close to the centre of the whole dataset, takes more iterations to converge

Forgy initialization

(Edward W. Forgy, 1965)

Pick k random datapoints as starting centroids.

- Pros: more natural spread across the dataspace
- Cons: susceptible to bad luck

k -means++

Modern standard, default in Scikit-learn.

Choose "far away but random" points - uses probability distribution proportional to square of distance to first randomly chosen point.

- Pros: reduces number of iterations to converge, outperforms random selection
- Cons: increased computational time for set-up

Manual initialization

Manually choose the starting centroids, if you have domain knowledge (ex. geographical locations).

- Pros: high accuracy
- Cons: requires prior knowledge, prone to human bias

Evaluation of clusters

Use cost (inertia) to compare clustering results.

$$C = \sum_i ||x_i - \mu(x_i)||^2$$

μ : associated centroid

Number of clusters k

- Choose k based on domain knowledge
- Try different values of k , choose based on elbow method

Downside of k -means

- Donut
- Two parallel lines
- Differently sized clusters

k -means overview

- Linear boundaries
- Point-to-centroid distance-based
- Takes number of clusters as parameter k
- Assumes circular clusters
- Sensitive to outliers
- Not suitable for non-convex shapes
- $O(nkl)$
 - n : number of samples
 - k : number of clusters
 - l : number of iterations

Table of Contents

- 1 Introduction to clustering
- 2 k -means
- 3 Agglomerative clustering**
- 4 DBSCAN
- 5 Clustering algorithms overview
- 6 Code example

Agglomerative clustering

Works bottom-up. Starts with n clusters, each datapoint for its own. Merges the closest pair of clusters until everything is in one cluster (or stopping criterion is met).

Agglomerative clustering visualized

Dendrogram

Dendrogram

Tree-like diagram visualizing the distances (dissimilarity) between clusters.

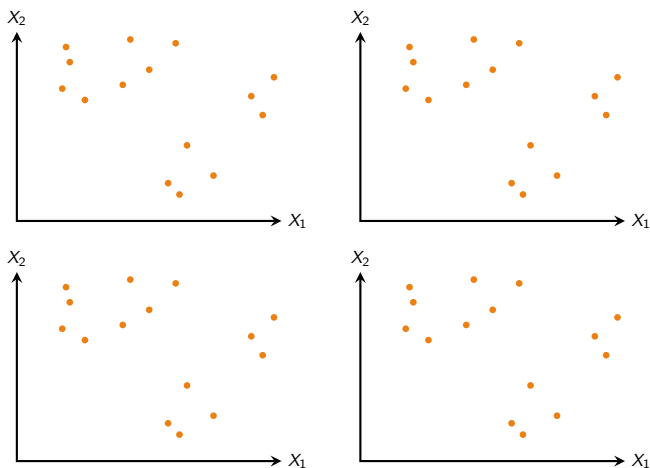
Cutting the dendrogram horizontally at a certain height: the number of intersects = the number of clusters.

Finding a good cut candidate: large gap between groups, find the longest vertical stretch.

Measure of distance between clusters

- **Single link:** distance between the closest points in the two clusters. Sensitive to outliers, might result in chaining (long thin clusters).
- **Complete link:** distance between the furthest points in the two clusters. Tends to find compact circular clusters.
- **Average link:** average distance between all pairs of points. Scales poorly.
- **Ward's link:** minimizes the increase in total within-cluster sum of squares (WCSS). Looks for dense and even clusters, might fail with weird shapes and different sizes (like k -means).

Measure of distance visualized



Agglomerative clustering overview

- Builds a tree structure: dendogram
- Pair-wise distance based
- No pre-defined number of clusters
- Captures nested relationships
- Multiple options for cluster distance metrics
- Can create non-spherical clusters, depending on the distance metric
- Sensitive to varying density, highly varying cluster size and elongated clusters
- Computationally expensive, $O(n^2)$

Table of Contents

- 1 Introduction to clustering
- 2 k -means
- 3 Agglomerative clustering
- 4 DBSCAN**
- 5 Clustering algorithms overview
- 6 Code example

DBSCAN

= Density-Based Spatial Clustering of Applications with Noise

Clusters are high-density regions separated by areas of low density. Allows for any shapes and helps identify noise (not belonging to any group).

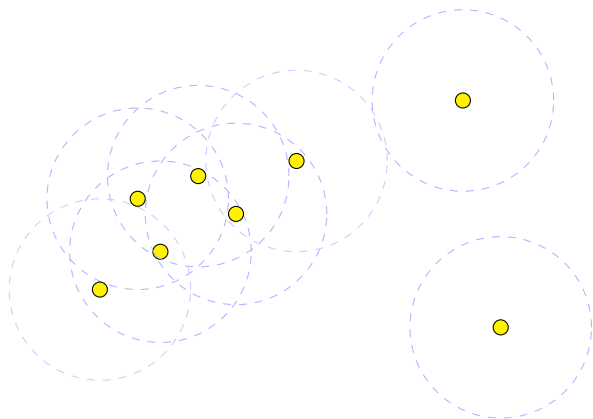
Dense region

Consider points within radius ϵ . A region is dense if it has at least n samples.

Each point in the dataset is:

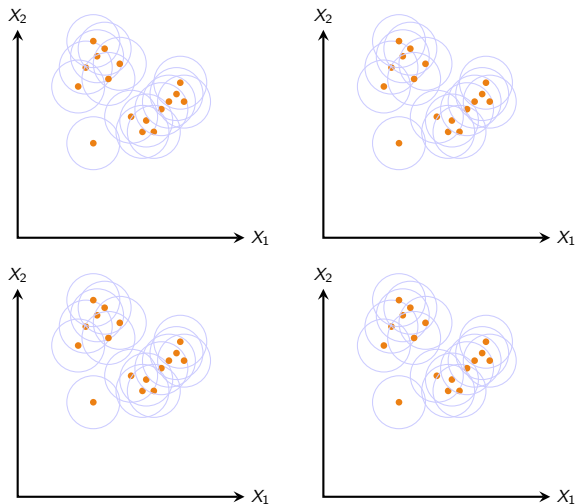
- **Core point:** a point with at least n points within radius ϵ , heart of the clusters.
- **Border point:** a point within ϵ from a core point but with $< n$ neighbours within its ϵ
- **Noise:** neither a core or border point. They get explicitly labelled (as -1).

Dense regions visualized



DBSCAN algorithm

- 1 Draw a circle of radius ϵ around every point.
- 2 If the circle contains at least n points, consider it a **core** point.
- 3 If the point is not a core point, but in the circle of a core point, it is a **border** point.
- 4 If it is neither, it is a **noise** point.
- 5 Remove noise points.
- 6 All core points reachable through each other's circles belong to the same cluster.
- 7 All border points are assigned to the cluster of the closest core point.

DBSCAN algorithm visualized, $n = 4$ 

Setting hyperparameters

n (MinPts)

- Noisy data: increase n to ignore groups of noise
- Depends on domain knowledge
- For most 2D datasets try $n = 4$

ϵ

- Too small: everything becomes noise
- Too large: everything is in one cluster
- Elbow method

Table of Contents

- 1 Introduction to clustering
- 2 k -means
- 3 Agglomerative clustering
- 4 DBSCAN
- 5 Clustering algorithms overview**
- 6 Code example

Comparison

Feature	<i>k</i> -means	Agglomerative	DBSCAN
Approach	Centroid-based	Hierarchical	Density-based
Parameters	k	k or threshold	ϵ , n
Noise handling	Sensitive	Integrated	Labelled -1
Complexity	$O(nkl)$	$O(n^2)$	$O(n \log n)$
Use Case	Well-separated blobs	Smaller datasets	Complex shapes, noise

Visual overview of algorithms

`https:`
`//scikit-learn.org/stable/modules/clustering.html`

Table of Contents

- 1 Introduction to clustering
- 2 k -means
- 3 Agglomerative clustering
- 4 DBSCAN
- 5 Clustering algorithms overview
- 6 Code example**

Code

Github link: https://github.com/304125/MAL1_S26/tree/master/11_Clustering